

Research Paper RAG Assistant: A Configurable Retrieval-Augmented Generation Pipeline for Scientific Document QA

ADSP 34002 IP09 – Capstone I

Spring 2026

Abstract

This report describes a configurable Retrieval-Augmented Generation (RAG) system built for question answering over a corpus of AI/ML research papers. The system supports multiple retrieval architectures, including dense retrieval, hybrid BM25+dense retrieval with reciprocal rank fusion (RRF), parent-child hierarchical retrieval, and a final parent-child hybrid architecture with cross-encoder reranking. The pipeline is implemented as a reproducible, configuration-driven Python project with modular ingestion, chunking, indexing, retrieval, generation, evaluation, and error analysis components. Across 496 benchmark queries, the best retrieval-only configuration achieved a Hit Rate@3 of 0.9919, while the final production-style parent-child hybrid rerank architecture achieved 0.9899 and provided richer parent-level context for grounded generation.

1. System Architecture

1.1. Objective

The goal is to build a complete RAG pipeline over a corpus of scientific PDFs. Given a user question, the system retrieves relevant evidence from the indexed research papers and optionally generates a grounded answer with inline citations. The assignment benchmark contains 75 AI/ML research papers and 496 query-answer pairs with document relevance labels.

1.2. End-to-End Pipeline

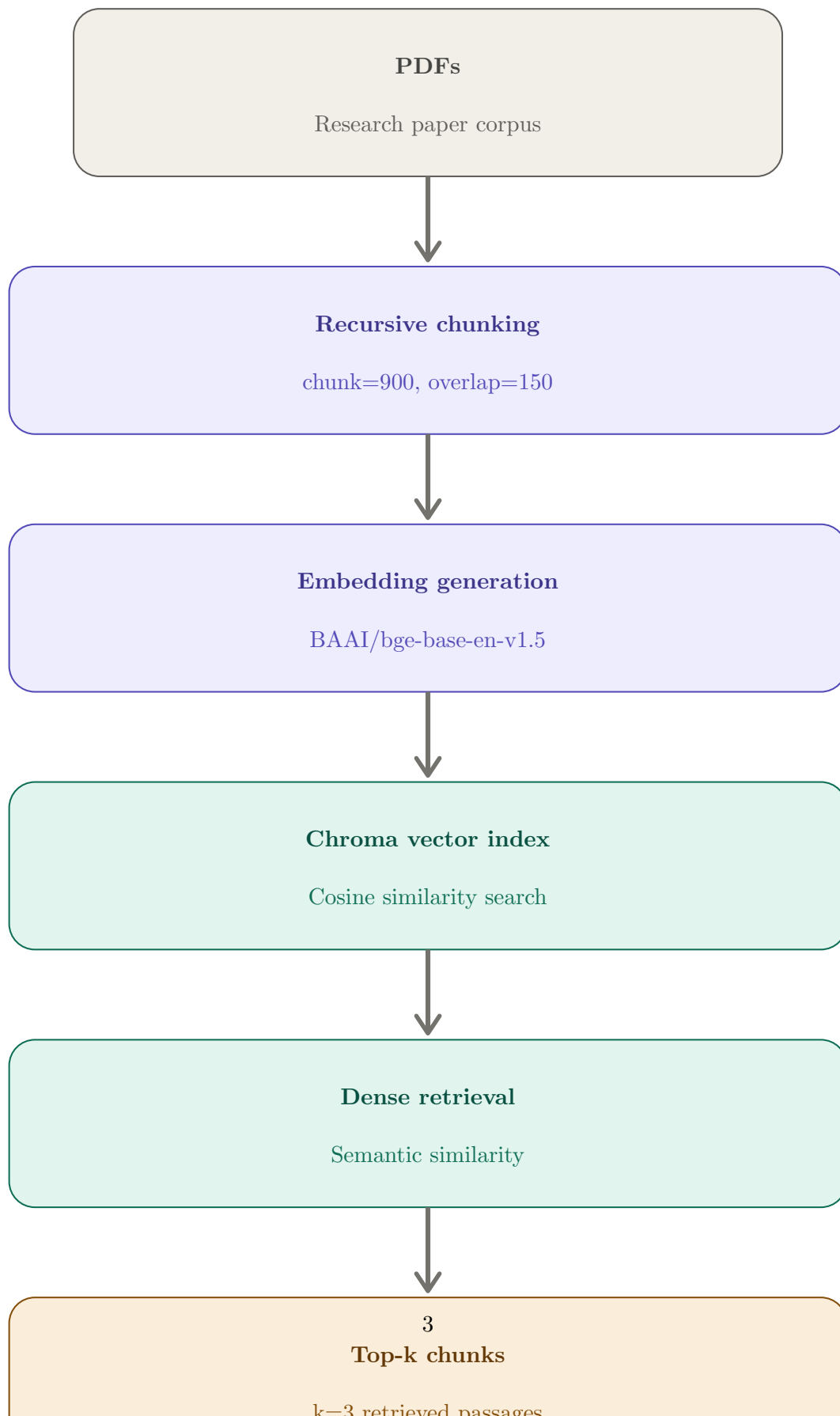
The system is configuration-driven through `config.yaml`. Each experiment uses the same shared pipeline but changes chunking, retrieval, and reranking parameters.

PDFs

- > PDF parsing and metadata extraction
- > Chunking: recursive or parent-child
- > Embedding generation
- > Vector indexing in ChromaDB
- > Optional BM25 indexing
- > Retrieval: dense or hybrid
- > Optional RRF fusion
- > Optional cross-encoder reranking
- > Optional grounded answer generation
- > Retrieval/RAGAS evaluation and error analysis

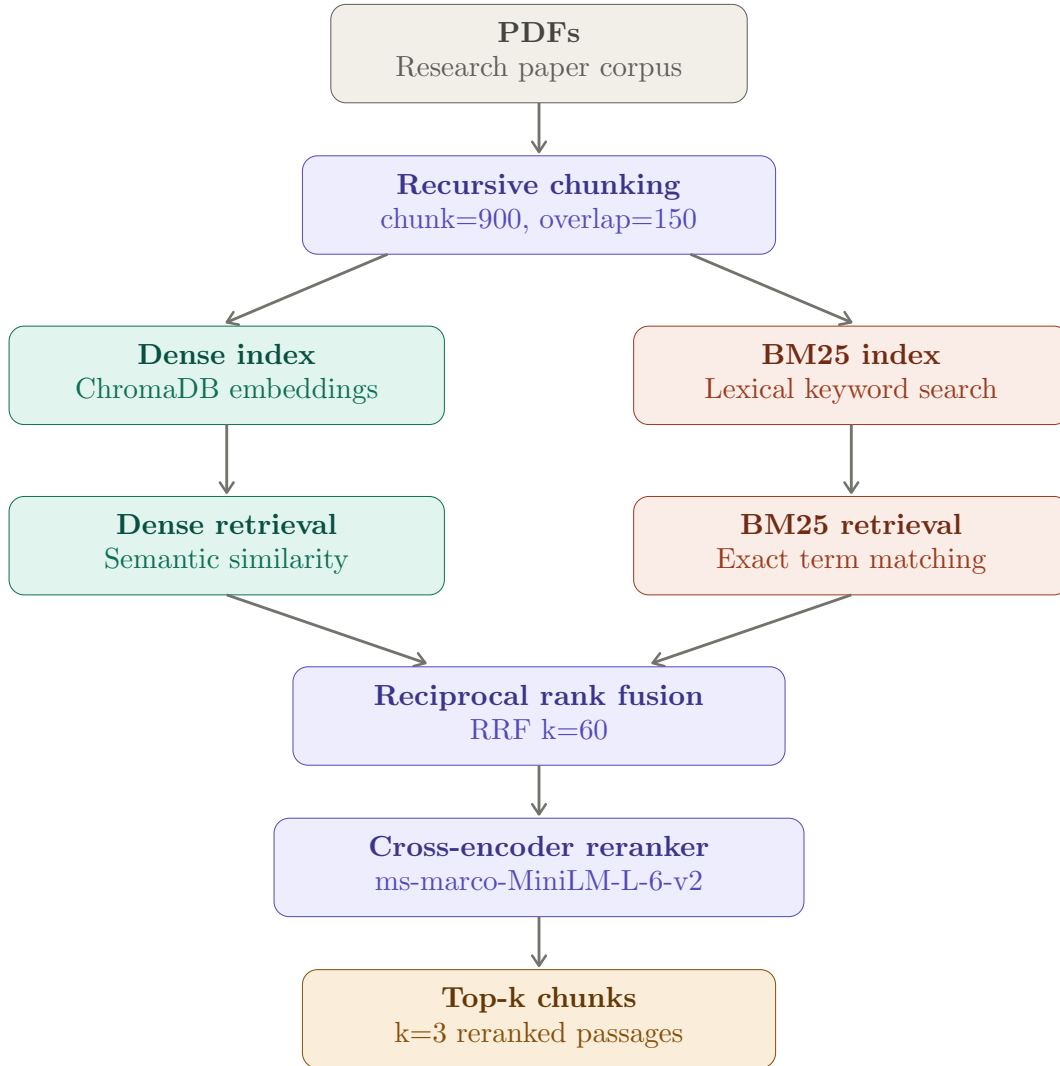
1.3. Implemented Architectures

A1. Baseline Dense RAG. A standard semantic retrieval baseline using recursive text chunks and ChromaDB vector search.

A1 — Baseline Dense RAG

A2. Hybrid BM25 + Dense + Reranker. A retrieval-focused architecture combining lexical and semantic search. BM25 captures exact terminology, dense retrieval captures semantic similarity, RRF merges rankings, and a cross-encoder reranker improves top-k precision.

A2 — Hybrid BM25 + Dense + Reranker



Hit Rate@3: 0.9919 (492/496) — highest retrieval score

Figure 2: Hybrid BM25 + Dense + Reranker Architecture

A3. Parent-Child Hierarchical RAG. A hierarchical architecture where small child chunks are indexed for precise retrieval and expanded into larger parent contexts for downstream generation.

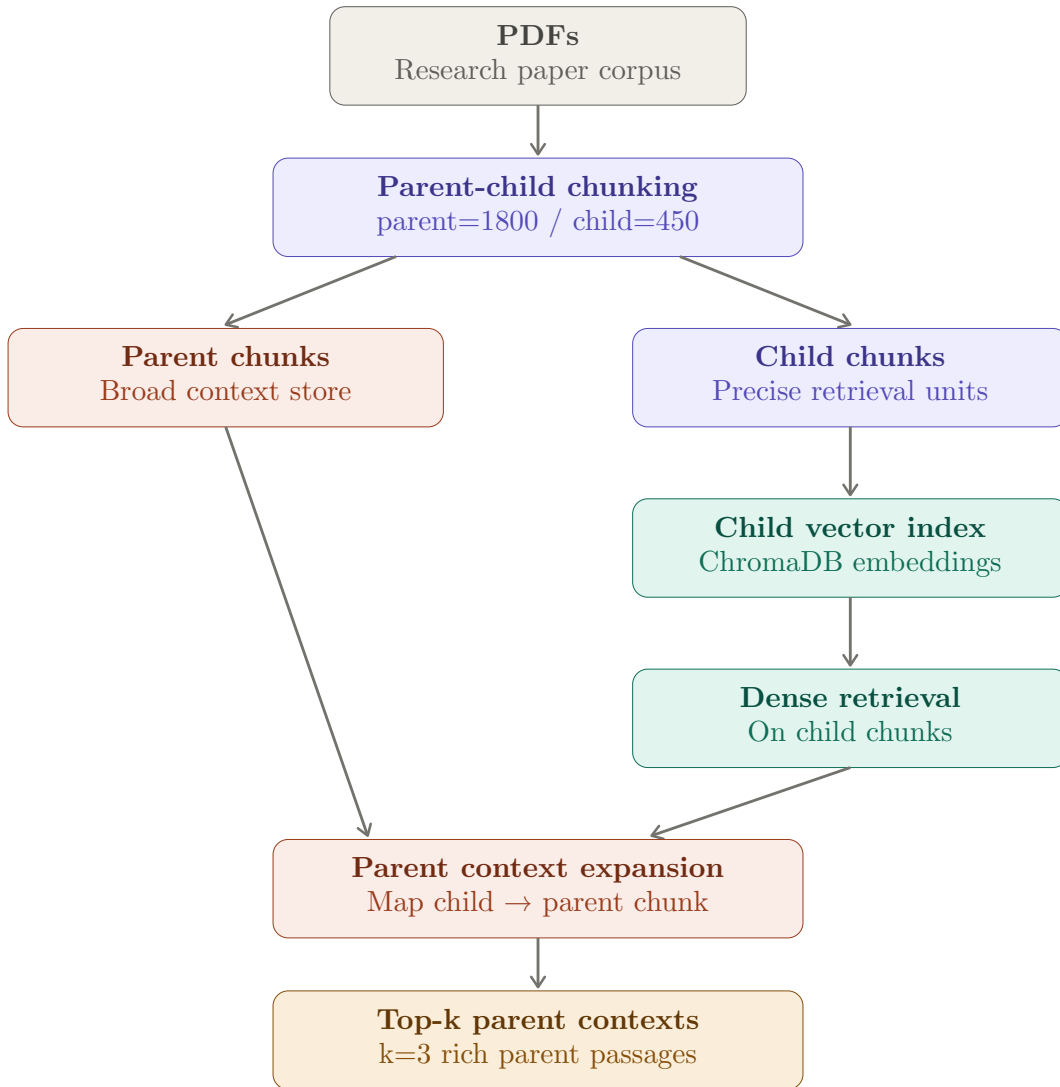
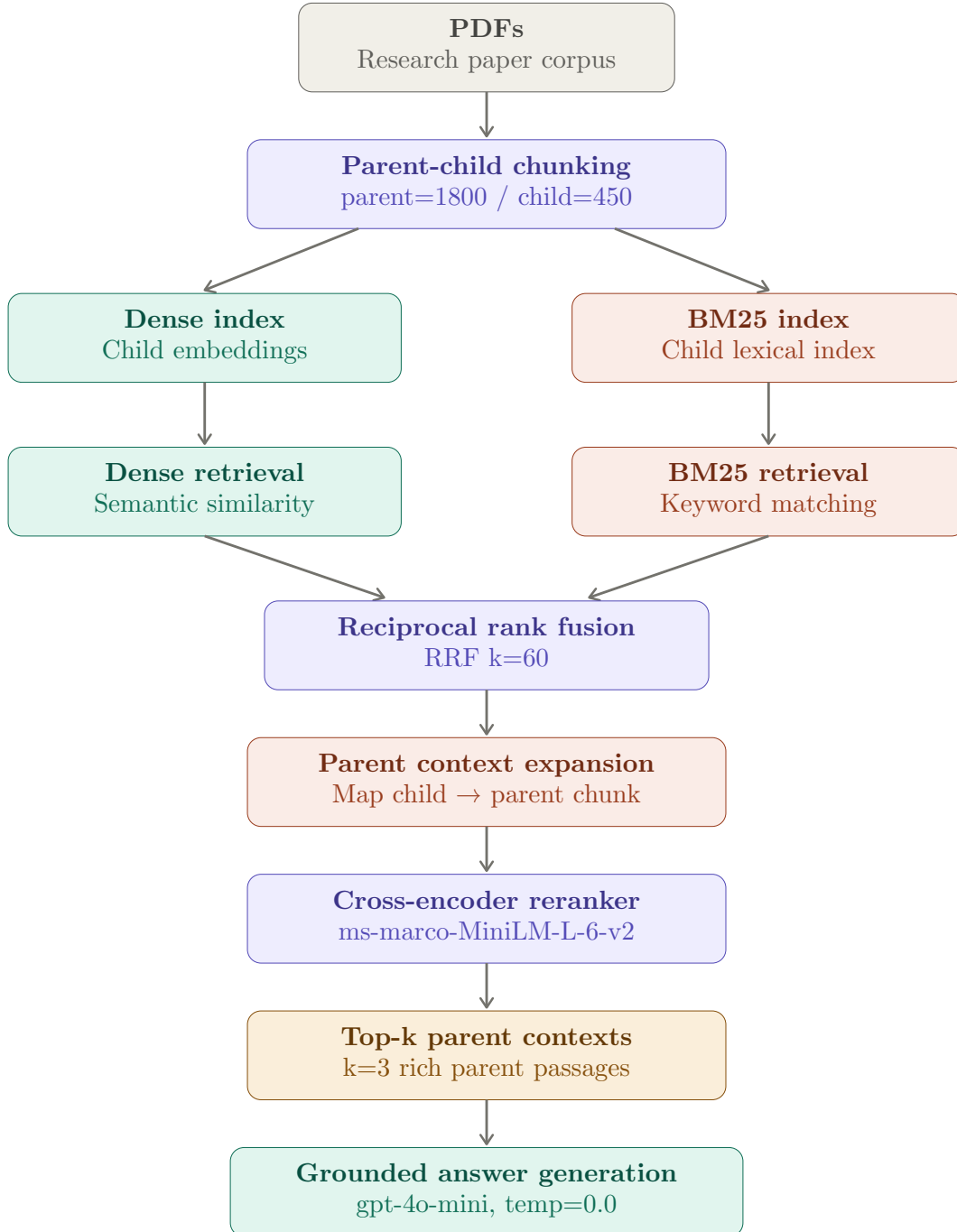
A3 — Parent-Child Hierarchical RAG

Figure 3: Parent-Child Dense RAG Architecture

A4. Final Recommended Architecture. The final architecture combines parent-child chunking, hybrid retrieval, RRF, parent expansion, and cross-encoder reranking.

A4 — Final Recommended Architecture

Parent-child + hybrid + RRF + reranker + grounded generation



Hit Rate@3: 0.9899 (491/496) — recommended production architecture

Figure 4: Final Recommended RAG Architecture

1.4. Technology Stack

Category	Tool / Library	Purpose
Language	Python	Core implementation
PDF parsing	PyMuPDF	Extract text and PDF metadata
Chunking	LangChain text splitters	Recursive and parent-child splitting
Embedding model	BAAI/bge-base-en-v1.5	Dense semantic embeddings
Vector database	ChromaDB	Persistent vector storage and retrieval
Keyword retrieval	rank_bm25	BM25 lexical search
Fusion	Reciprocal Rank Fusion	Combine dense and BM25 rankings
Reranker	cross-encoder/ms-marco-MiniLM-L-6-v2	Query-context relevance scoring
Generation	OpenAI/Groq SDKs	Grounded answer generation
Evaluation	Hit Rate@3, RAGAS	Retrieval and answer-quality evaluation
Configuration	YAML	Reproducible experiment control

Table 1: Core technology stack.

1.5. Key Hyperparameters

Component	Configuration
Embedding model	BAAI/bge-base-en-v1.5, normalized embeddings
Recursive chunking	chunk size 900, overlap 150
Parent chunking	parent size 1800, overlap 250
Child chunking	child size 450, overlap 75
Dense retrieval	cosine similarity via ChromaDB
Hybrid retrieval	dense + BM25 using RRF
RRF constant	60
Retrieval output	top_k = 3
Reranker candidate pool	fetch_k = 20
Reranker output	top_n = 3
Generation LLM	gpt-4o-mini or Groq-compatible model
Generation temperature	0.0

Table 2: Primary hyperparameters used in the configurable pipeline.

2. Design Decisions

2.1. PDF Parsing and Preprocessing

PyMuPDF was selected for PDF parsing because it is lightweight, fast, and easy to integrate into a local Python pipeline. The parser extracts per-page text, document metadata, title, author fields when available, page count, and an arXiv-style document identifier from the filename. The current implementation primarily relies on text extraction. Table content is preserved when it appears in the extracted text stream, but full structural table extraction into Markdown is treated as a planned extension. Full image understanding is not implemented in the current run.

2.2. Chunking Strategy

Two chunking strategies were implemented. Recursive chunking provides a simple baseline and supports fair comparison across dense and hybrid retrieval. Parent-child chunking is used to balance retrieval precision and generation context. Child chunks are small enough to retrieve precisely, while parent chunks provide broader context to the generator.

2.3. Embedding and Vector Store

The embedding model `BAAI/bge-base-en-v1.5` was selected because it offers strong open-source semantic retrieval performance while avoiding paid embedding costs. ChromaDB was chosen as the vector database because it supports persistent local indexes, metadata storage, cosine search, and simple local development without external infrastructure.

2.4. Retrieval Strategy

Dense retrieval is used as the required baseline. BM25 is added because scientific papers contain exact identifiers such as model names, datasets, equations, metrics, and acronyms that dense retrieval may blur. RRF is used to combine dense and BM25 ranked lists without needing to normalize incompatible score scales. The cross-encoder reranker then directly scores query-context pairs to improve final top-k precision.

2.5. Grounded Prompting

The generation prompt places retrieved context before the question, labels each chunk as a source, requires the model to answer only from provided context, and includes an explicit fallback: “I do not have enough information in the provided context to answer this question.” This design reduces hallucination risk and makes generated answers auditable.

3. Experimental Results

3.1. Benchmark Setup

All retrieval experiments were evaluated on 496 benchmark queries. A retrieval hit is counted when the correct source paper from `qrels.json` appears in the top 3 retrieved results. The query distribution was: 329 text-only, 115 text-image, 28 text-table, and 24 text-table-image queries.

3.2. Retrieval Results

Experiment	Architecture	Hits	Hit Rate@3
A1	Baseline Dense RAG	481 / 496	0.9698
A2	Hybrid BM25 + Dense + RRF + Reranker	492 / 496	0.9919
A3	Parent-Child Dense RAG	481 / 496	0.9698
A4	Parent-Child + Hybrid + RRF + Reranker	491 / 496	0.9899

Table 3: Hit Rate@3 comparison across four retrieval configurations.

A2 achieved the highest retrieval-only score. A4 was slightly lower by one query but is recommended as the production-style architecture because it returns richer parent-level context while maintaining near-best retrieval accuracy.

3.3. A4 Modality Breakdown

Query source	Hits	Total	Hit Rate@3
Text	327	329	0.9939
Text-image	112	115	0.9739
Text-table	28	28	1.0000
Text-table-image	24	24	1.0000

Table 4: A4 retrieval performance by query modality.

The lower score for text-image queries suggests that image-dependent questions remain a key limitation because the current implementation does not perform full figure captioning or vision-language analysis.

3.4. Generation and RAGAS Pilot

Metric	Score
Faithfulness	0.7659
Answer relevancy	0.9459
Context precision	0.7667
Context recall	0.7000
Answer correctness	0.6097

Table 5: A4 RAGAS pilot evaluation on 5 generated answers.

4. Error Analysis

4.1. A4 Retrieval Failures

Query ID	Source	Question	Failure Type
d170153a	text	What is tokenization in the context of large language models?	Retrieval error
5c045887	text-image	What role do cues play in guiding behavior within maze experiments?	Retrieval error
42baf452	text	How is job data structured in relation to training, validation, and test sets?	Retrieval error
8666492a	text-image	What are some effective strategies to defend against adversarial behavior in AI models?	Retrieval error

Query ID	Source	Question	Failure Type
4a74ec93	text-image	Which AI model most frequently uses legal terminology in its prompts?	Retrieval error

Table 6: A4 failed retrieval cases. Query IDs are shortened for readability.

5. Limitations and Future Work

5.1. Known Limitations

- Full image understanding is not implemented.
- Multimodal support currently focuses on text and table-style content.
- RAGAS evaluation was performed on a pilot sample due to API cost constraints.
- Parent-child expansion may introduce broader context and slightly affect ranking precision.
- PDF parsing quality depends on the structure and formatting of individual papers.

5.2. Future Work

A5. HyDE + Multi-Query Expansion. Future work will add HyDE-based query generation, multi-query expansion, retrieval diversification, and improved recall for ambiguous scientific questions.

A6. Multimodal Layer. Future work will add table-aware extraction, figure caption extraction, OCR-based image text extraction, and modality-aware retrieval.

Streamlit Interface. A planned Streamlit UI will allow users to upload papers, select an architecture, ask questions interactively, compare retrieved chunks, inspect citations, and view evaluation metrics.

6. Conclusion

The project implements a modular and reproducible RAG pipeline for research paper question answering. The highest retrieval score was achieved by the hybrid BM25+dense retrieval system with RRF and reranking. The final recommended architecture, A4, combines parent-child context expansion with hybrid retrieval and reranking, achieving 0.9899 Hit Rate@3 while providing richer context for grounded generation. The main remaining improvements are full multimodal processing, broader RAGAS evaluation, and an interactive UI.

A. Appendix A: Reproducibility

A.1. Repository Requirements

The submission repository should include:

```
README.md
requirements.txt
config.yaml
run_pipeline.py
run_error_analysis.py
src/
results.json
```

A.2. Expected Data Layout

```
data/raw/
  pdfs/
  queries.json
  answers.json
  qrels.json
```

A.3. Primary Run Commands

```
pip install -r requirements.txt
python run_pipeline.py
python run_error_analysis.py
```

B. Appendix B: Configuration Summary

```
pipeline:
  run_ingestion: false
  run_chunking: false
  run_indexing: false
  run_retrieval_eval: true
  run_generation: false
  run_ragas_eval: false

retrieval:
  top_k: 3
  fetch_k: 20
  hybrid:
    fusion: rrf
    rrf_k: 60

models:
  embedding:
    name: BAAI/bge-base-en-v1.5
  reranker:
    name: cross-encoder/ms-marco-MiniLM-L-6-v2
```

C. Appendix C: Results JSON Summary

```
A1_baseline_dense: hit_rate_at_3 = 0.9698, hits = 481/496
A2_hybrid_reranker: hit_rate_at_3 = 0.9919, hits = 492/496
A3_parent_child: hit_rate_at_3 = 0.9698, hits = 481/496
A4_parent_child_hybrid_rerank: hit_rate_at_3 = 0.9899, hits = 491/496
```

D. Appendix D: Cost Controls

For cost-sensitive runs, generation and RAGAS should be disabled until retrieval is validated. Generation and RAGAS can then be run on a small sample before scaling.

```
generation:
  sample_size: 5

evaluation:
  ragas_sample_size: 5
```